



WYDZIAŁ ELEKTRONIKI I TECHNIK INFORMACYJNYCH

SYSTEMY CYFROWE

Projektowanie wielostopniowego układu przetwarzania sygnałów wizualnych

Mateusz Kokoszka	Szymon Domański	Kuba Kulczyński	Jacek Cieszewski
343116	343095	343121	343086

7 czerwca 2026

Wersja	Data	Autor	Opis Zmian
1.0	16.03.2026	MK	Opis Etapu 1, Utworzenie krótkiego wprowadzenia do zadania
1.1	17.03.2026	MK	Poprawki w opisie Etapu 1, dokładniejszy opis narzędzi oraz podział na role
1.2	17.03.2026	MK	Dodanie dodatkowych informacji na temat metod zarządzania projektem
2.1	02.04.2026	MK	Pierwsze opisywanie etapu okdrywania
3.1	25.04.2026	MK	Raport z III etapu
3.2	04.05.2026	MK	Ostatnie poprawki związane z III etapem
5.1	07.06.2026	MK	Raport z V etapu

Tabela 1: Historia zmian

Spis treści

1	Wprowadzenie do projektu	3
1.1	Kontekst historyczny i sytuacyjny	3
1.2	Robot inspekcyjny „Wędrowiec”	3
1.3	Cel zadania	3
2	Realizacja	4
2.1	Etap I - Organizacja prac	4
2.1.1	Skład zespołu	4
2.1.2	Design Thinking	4
2.1.3	Metody zarządzania projektem	5
2.1.4	Narzędzia	5
2.1.5	Ryzyka i założenia	5
2.1.6	Harmonogram prac / wykres Gantt	6
2.2	Etap II - Odkrywanie	7
2.2.1	Przegląd literatury	7
2.2.2	Przegląd istniejących rozwiązań	7
2.2.3	Definicja problemu technicznego	10
2.2.4	Analiza 5x Why	11
2.3	Etap III - Definiowania	12
2.3.1	Dyskusja rozwiązań, ich wady i zalety oraz wybór finalny	12
2.3.2	Analiza wariantów architektury	12
2.3.3	Uzasadnienie wyboru i bariery techniczne	13
2.3.4	Schemat blokowy systemu – koncepcja i interfejsy	13
2.3.5	Omówienie kodu koncepcyjnego python	14
2.3.6	Szczegółowy opis działania skryptu	14
2.3.7	Ekstrakcja cech zorientowana na sprzęt (Hardware-Aware)	14
2.3.8	Augmentacja danych i trening	15
2.3.9	Konwersja Fixed-Point Q8	15
2.3.10	Generowanie pakietu Verilog	15
2.3.11	Kod źródłowy skryptu eksportującego	15
2.3.12	Wnioski	17
2.3.13	Scenariusze testowe modelu uczącego	17
2.3.14	Implementacja testów weryfikacyjnych	18

2.4	Etap rozwijania (Develop)	22
2.4.1	Specyfikacja projektu HDL	22
2.4.2	Szczegółowa implementacja sprzętowa modelu sieci neuronowej	23
2.4.3	Deklaracja sygnałów i architektura FSM	23
2.4.4	Etap wczesnej ekstrakcji cech (Stan S_FEATURES)	23
2.4.5	Przetwarzanie warstwy wyjściowej W2 (Stan S_OUTPUT)	24
2.4.6	Pakiet pamięci stałych (wagi_mlp_pkg.vhd)	25
2.4.7	Plan weryfikacji funkcjonalnej	25
2.4.8	Plan weryfikacji funkcjonalnej i symulacje czasowe	26
2.4.9	Implementacja skryptu testowego	26
2.4.10	Omówienie kodu i analiza mechanizmu weryfikacji	27
2.4.11	Mapowanie klas wyjściowych (Dekodowanie sygnału)	27
2.4.12	Symulacje	28
2.4.13	Rezultaty implementacji	29
2.4.14	Ocena prototypu	30
2.5	Etap dostarczania (Deliver) i ocena końcowa	31
2.5.1	Implementacja i środowisko weryfikacyjne	31
2.5.2	Parametry konfiguracyjne i instrukcja uruchomieniowa (Ad b)	32
2.5.3	Generacja pseudolosowych wektorów testowych (LFSR)	34
2.5.4	Testy wydajnościowe i parametry czasowe	35
2.5.5	Analiza systemu jako całości (Ad d)	35
2.5.6	Wnioski, ograniczenia i kierunki rozwoju	36
3	Podsumowanie	37
3.1	Czy projekt przebiegał zgodnie z planem?	37
3.2	Napotkane problemy i zastosowane rozwiązania	37
3.3	Wniosek ostateczny	37
	Literatura	39

1 Wprowadzenie do projektu

1.1 Kontekst historyczny i sytuacyjny

Rok 2056. Legendarna stacja *Babylon 5*, niegdyś kluczowe centrum dyplomatyczne galaktyki, została całkowicie opuszczona. Po wyłączeniu systemów podtrzymywania życia i ewakuacji personelu, obiekt dryfuje w martwej strefie przestrzeni kosmicznej, pełniąc funkcję gigantycznego, metalowego sarkofagu.

Misja badawcza, w której uczestniczymy, zakłada odzyskanie tzw. „Czarnej Skrzynki” danych. Zawiera ona kluczowe informacje niezbędne do odrodzenia rasy G’Kar. Zadanie jest utrudnione przez fakt, że główne systemy sterowania stacji są nieaktywne, a korytarze prowadzące do Sektora Dowodzenia zostały zablokowane przez osunięcia konstrukcyjne oraz wycieki promieniowania.

1.2 Robot inspekcyjny „Wędrowiec”

Do eksploracji niebezpiecznych sekcji stacji wyznaczony został „Wędrowiec” – autonomiczny robot inspekcyjny z czasów świetności obiektu. Robot charakteryzuje się specyficzną budową systemów percepcji:

- Posiada prymitywny sensor wizualny w postaci matrycy czarno-białej o rozdzielczości 8×8 pikseli, dzięki któremu może odczytywać stare, fotoluminescencyjne znaki nawigacyjne namalowane na ścianach korytarzy.
- Musimy zaprojektować system, który musi interpretować cztery komendy odczytywane z matrycy w postaci strzałek: ruch naprzód, skręt w prawo, skręt w lewo oraz zawrócenie (oznaczone znakiem X).

1.3 Cel zadania

Głównym celem projektu jest zaprojektowanie od nowa „mózgu” robota, czyli wielostopniowego układu przetwarzania sygnałów wizualnych. System musi radzić sobie z niestabilnymi warunkami oświetleniowymi oraz niedoskonałościami wzorców (różna grubość linii, przekrzywienia, przesunięcia na matrycy).

Projektowany układ musi składać się z trzech kluczowych warstw:

1. **Layer 1 (Ekstrakcja cech):** Redukcja danych z matrycy 8×8 do postaci wektora cech (tzw. matematycznego „odcisku palca” kształtu), odpornego na translacje i szum czujnika.
2. **Layer 2 (Adaptacyjne wnioskowanie):** Struktura zdolna do generalizacji i uczenia się na zbiorze treningowym, zamiast sztywnych reguł programistycznych.
3. **Layer 3 (Moduł decyzyjny):** Ostateczna klasyfikacja komendy na podstawie analizy sekwencji 6 następujących po sobie klatek obrazu.

2 Realizacja

2.1 Etap I - Organizacja prac

2.1.1 Skład zespołu

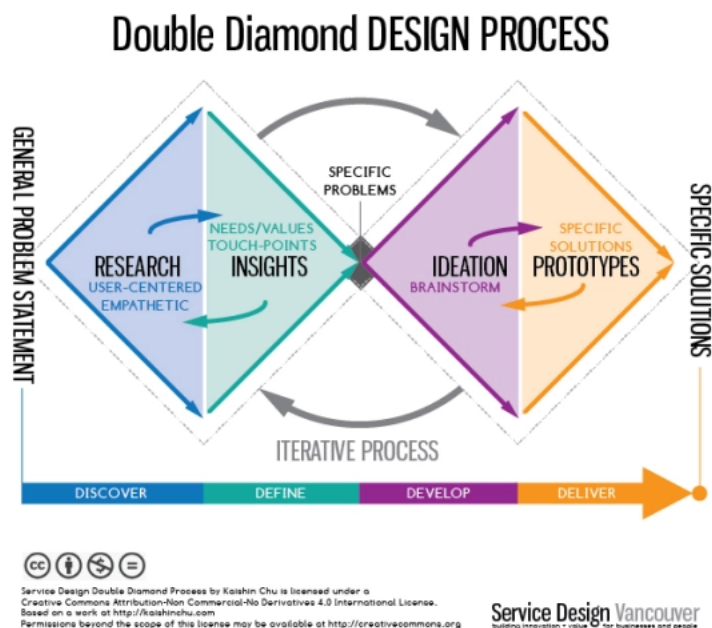
- **Mateusz Kokoszka:** Lider zespołu, Odpowiedzialny za dokumentację,
- **Szymon Domański:** Odpowiedzialny za kod
- **Kuba Kulczyński:** Odpowiedzialny za testy
- **Jacek Cieszewski:** Odpowiedzialny za kod

Lider zespołu został wybrany podczas krótkiej konwersacji dokonanej w aplikacji Messenger.

Zasady pracy zespołowej: Wewnątrz zespołu umówiliśmy się na cotygodniowe dwu godzinne spotkania we wtorki w trakcie okienka, w razie problemów informacje oraz uwagi będą przesyłane przy użyciu aplikacji Messenger.

2.1.2 Design Thinking

W projekcie wykorzystujemy model **Double Diamond** [1], który strukturyzuje proces projektowy w czterech fazach:



1. **Faza I:** Research związany z problemem, identyfikacja problemu oraz istniejących przykładowych rozwiązań,
2. **Faza II:** Analiza zebranych informacji, wyciągnięcie najważniejszych wniosków,
3. **Faza III:** Burza mózgów, wymyślanie wstępnych rozwiązań,

4. **Faza IV:** Doskonalenie wniosków wyciągniętych z burzy mózgów, tworzenie prototypowych rozwiązań oraz ich testowanie

2.1.3 Metody zarządzania projektem

Rozpatrywaliśmy kilka metod zarządzania projektem takich jak np.:

- **Waterfall** - liniowy model słabo sprawdzający się niespodziewanych zmianach w trakcie pracy nad projektem
- **Scrum** - metoda polegająca na krótkich sprintach (kilku tygodniowych) pozwalających na bieżące dostosowywanie zakresu i szybką weryfikację postępu

Ostatecznie, jako metodę zarządzania projektem wybraliśmy metodę **Kanban** charakteryzującą się tabelą zawierającą: zadania do wykonania, zadania w trakcie wykonywania zadania wykonane. Tabela utworzona została w repozytorium w gitlabie.

2.1.4 Narzędzia

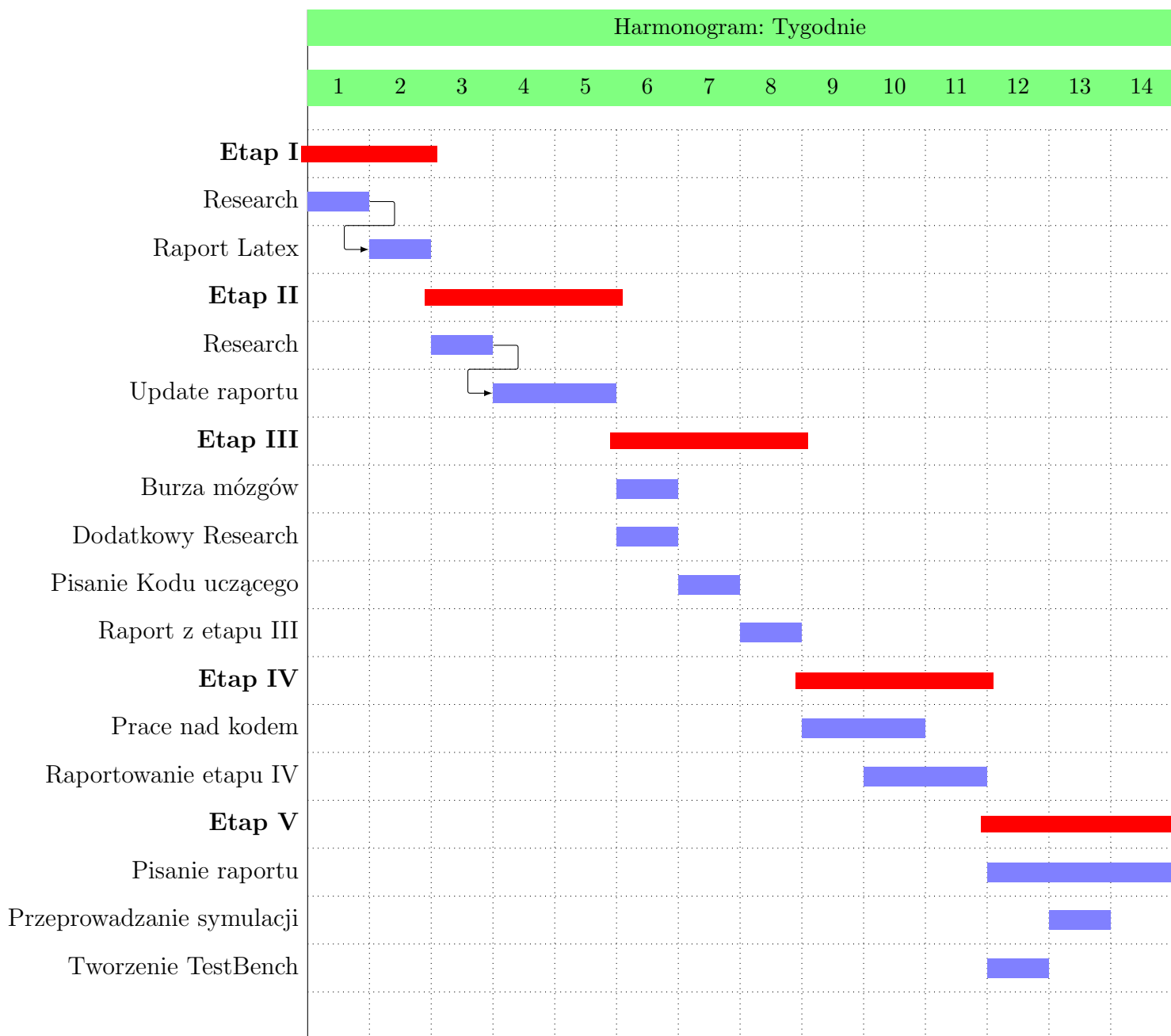
Narzędzia z których korzystamy:

- **Messenger** - komunikacja wewnątrz zespołu, najwygodniejsza i codziennie używana przez nas opcja,
- **Discord** - komunikacja wewnątrz zespołu, spotkania online, wybraliśmy ten komunikator ze względu na to że każdy z nas ma w nim duże doświadczenie oraz długi staż użytkowania,
- **Overleaf [2]** - tworzenie raportu w LATEX, narzędzie w którym mamy pewne doświadczenie oraz jedno z lepszych i popularniejszych rozwiązań przy tworzeniu raportów,
- **Gitlab.pw** - zamieszczanie wszystkich potrzebnych plików, wybrany został gitlab.pw, ponieważ mamy z nim styczność od pierwszego semestru i mamy lepsze pojęcie na temat obchodzenia się z nim,
- **Github.com [3]** - publiczne repozytorium z najważniejszymi plikami i poradnikami zostało stworzone na Github.com, ponieważ jest to bardzo wygodne rozwiązanie,
- **Pycharm/python** - tworzenie zarysu oraz wstępnych koncepcji systemu przetwarzania sygnałów wizualnych w pythonie, wybraliśmy Pycharma ponieważ korzystaliśmy z niego w pierwszym semestrze i jest to bardzo wygodne zintegrowane środowisko programistyczne,
- **Intel Quartus Prime [4]** - Synteza HDL i symulacja pracy układu na FPGA, dobór tego narzędzia też został dokonany na podstawie styczności z nim podczas pierwszego semestru studiów.

2.1.5 Ryzyka i założenia

Głównym ryzykiem projektowym jest potencjalna trudność w odwzorowaniu funkcji z Pythona do Veriloga, co może nie przynieść oczekiwanych rezultatów w sprzęcie mimo poprawnej symulacji. Zakładamy wykorzystanie platformy FPGA jako docelowego środowiska uruchomieniowego.

2.1.6 Harmonogram prac / wykres Gantt



Rysunek 1: Wykres Gantt / harmonogram prac

2.2 Etap II - Odkrywanie

2.2.1 Przegląd literatury

2.2.2 Przegląd istniejących rozwiązań

- **Niezmienniki momentowe Hu (Hu moments)** Hu moments to zestaw siedmiu liczb, które opisują kształt obiektu na obrazie w sposób niezależny od jego położenia, wielkości i obrotu.
 - **Jak to działa?** Zanim otrzymamy 7 momentów Hu, algorytm musi przejść przez proces „oczyszczania” danych geometrycznych:

- **Momenty Surowe (Momenty Geometryczne)**

Momenty surowe (m_{pq}) mierzą pozycję pikseli na obrazie ważoną ich intensywnością, obliczaną za pomocą wzoru:

$$m_{pq} = \sum_x \sum_y x^p y^q I(x, y)$$

Gdzie:

- m_{pq} – moment geometryczny rzędu $(p + q)$,
 - x, y – współrzędne (pozycje) pikseli w obrazie,
 - p, q – rzędy momentu względem odpowiednio osi x oraz y ,
 - $I(x, y)$ – wartość intensywności (jasności) piksela w punkcie (x, y) .
- **Niezmienniki Hu (Momenty Hu)** Aby uzyskać odporność na translację (przesunięcie), stosujemy momenty centralne. Kluczem jest wyznaczenie środka ciężkości obiektu $(\bar{x}, \bar{y})$:

$$\bar{x} = \frac{\sum \sum x \cdot f(x, y)}{\sum \sum f(x, y)}, \quad \bar{y} = \frac{\sum \sum y \cdot f(x, y)}{\sum \sum f(x, y)}$$

Obliczając momenty względem tego środka ($\mu_{pq} = \sum \sum (x - \bar{x})^p (y - \bar{y})^q f(x, y)$), otrzymujemy wartości, które są „identyczne” niezależnie od położenia obiektu na matrycy. System redukuje 64 piksele do wektora 7 liczb opisujących geometrię kształtu.

- **Niezmienniki momentowe Hu**

Siedem niezmienników Hu ($h_1, \dots, h_7$) oblicza się z momentów centralnych znormalizowanych η_{pq} . Wzory na poszczególne składowe wektora cech prezentują się następująco:

$$\begin{aligned} h_1 &= \eta_{20} + \eta_{02} \\ h_2 &= (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2 \\ h_3 &= (\eta_{30} - 3\eta_{12})^2 + (3\eta_{21} - \eta_{03})^2 \\ h_4 &= (\eta_{30} + \eta_{12})^2 + (\eta_{21} + \eta_{03})^2 \\ h_5 &= (\eta_{30} - 3\eta_{12})(\eta_{30} + \eta_{12})[(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] \\ &\quad + (3\eta_{21} - \eta_{03})(\eta_{21} + \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2] \\ h_6 &= (\eta_{20} - \eta_{02})[(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2] + 4\eta_{11}(\eta_{30} + \eta_{12})(\eta_{21} + \eta_{03}) \\ h_7 &= (3\eta_{21} - \eta_{03})(\eta_{30} + \eta_{12})[(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] \\ &\quad - (\eta_{30} - 3\eta_{12})(\eta_{21} + \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2] \end{aligned}$$

Gdzie:

- m_{pq} – **Momenty surowe**: Liczby p i q określają ich rząd. Na przykład m_{00} to po prostu pole powierzchni obiektu (liczba białych pikseli).
 - $I(x, y)$ – **Jasność piksela**: Wartość w danym punkcie (x, y) . Dla obrazów czarno-białych wynosi 0 (tło) lub 1 (obiekt).
 - x, y – **Współrzędne**: Pozycja konkretnego piksela wiersz po wierszu i kolumna po kolumnie.
 - μ_{pq} – **Moment centralny**: Wartość obliczona po odjęciu współrzędnych środka ciężkości. Dzięki temu wynik jest taki sam, niezależnie od tego, w którym miejscu zdjęcia znajduje się obiekt (**niezależność od przesunięcia**).
 - η_{pq} (*eta*) – **Moment znormalizowany**: Powstaje przez podzielenie momentu centralnego przez pole powierzchni. Sprawia to, że wynik nie zmienia się, gdy obiekt jest powiększany lub pomniejszany (**niezależność od skali**).
 - γ (*gamma*) – **Współczynnik skalujący**: Pomocnicza liczba obliczana jako $\frac{p+q}{2} + 1$, która dba o to, by dzielenie przez pole powierzchni było matematycznie poprawne dla danego rzędu.
- **Jak to działa tak dokładnie?** Surowe wartości momentów H_u (h_i) są często bardzo małe i różnią się o wiele rzędów wielkości. Aby móc je efektywnie porównywać, stosuje się transformację logarytmiczną:

$$H_i = -\text{sign}(h_i) \cdot \log_{10}(|h_i|)$$

Tak przygotowany wektor cech wykorzystuje się do rozpoznawania obiektów poprzez obliczenie odległości (podobieństwa) między dwoma kształtami A i B :

$$I(A, B) = \sum_{i=1}^7 \left| \frac{1}{H_i^A} - \frac{1}{H_i^B} \right|$$

Im mniejsza wartość wskaźnika $I(A, B)$, tym większe prawdopodobieństwo, że badane kształty są identyczne, niezależnie od ich obrotu czy skali.

- **Charakterystyka Eulera** Charakterystyka Eulera (E) jest niezmiennikiem topologicznym, który pozwala na szybką klasyfikację kształtów na podstawie ich spójności i liczby otworów:

$$E = C - H$$

Gdzie:

- C – liczba spójnych składowych (osobnych obiektów),
- H – liczba otworów (dziur) wewnątrz obiektów.

Wartość E jest całkowicie niezależna od obrotu, skali oraz transformacji afinicznych obiektu, co czyni ją idealnym parametrem dla systemów o niskiej rozdzielczości matryc.

- **Detekcja Kierunku Strzałki na Podstawie Asymetrii Rozkładu Przestrzennego** Rozpoznawanie zwrotu strzałki w analizie obrazów binarnych opiera się na badaniu asymetrii rozkładu pikseli. Z topologicznego punktu widzenia strzałka charakteryzuje się cienkim rdzeniem oraz szerokim grotem, co prowadzi do znacznej dysproporcji w lokalnej gęstości obiektu. Algorytm wykorzystuje tę właściwość w trzech głównych etapach obliczeniowych.

- Wyznaczenie środka geometrycznego ramki otaczającej

W pierwszej fazie algorytm izoluje obiekt, wyznaczając najmniejszy prostokąt ograniczający (ang. *bounding box*), opisany współrzędnymi skrajnymi: $x_{min}, x_{max}, y_{min}, y_{max}$. Środek geometryczny tego obszaru, oznaczony jako punkt O , definiuje się następująco:

$$O_x = \frac{x_{min} + x_{max}}{2}, \quad O_y = \frac{y_{min} + y_{max}}{2}$$

Punkt O stanowi idealny środek układu odniesienia, niezależny od wewnętrznej struktury i ułożenia pikseli.

- Wyznaczenie centroidy (środka ciężkości)

Następnie obliczany jest rzeczywisty środek ciężkości obiektu, oznaczony jako punkt C . Wykorzystuje się do tego momenty przestrzenne (stosunek momentów rzędu pierwszego do momentu rzędu zerowego). Dla obrazu binarnego $I(x, y)$:

$$C_x = \frac{\sum_x \sum_y x \cdot I(x, y)}{\sum_x \sum_y I(x, y)}, \quad C_y = \frac{\sum_x \sum_y y \cdot I(x, y)}{\sum_x \sum_y I(x, y)}$$

Ze względu na wyższą koncentrację pikseli w obszarze grotu, punkt C ulega przesunięciu w jego kierunku względem punktu O .

- Analiza wektora przesunięcia

Kluczowym elementem metody jest wyznaczenie wektora przesunięcia $\vec{v}$ między środkiem ciężkości a środkiem geometrycznym:

$$\vec{v} = [\Delta x, \Delta y] = [C_x - O_x, C_y - O_y]$$

Składowe wektora $\vec{v}$ determinują orientację i zwrot strzałki (przy standardowym założeniu, że oś Y rośnie w dół obrazu):

- **Dominująca oś (orientacja):**

* $|\Delta x| > |\Delta y| \implies$ orientacja pozioma.

* $|\Delta y| > |\Delta x| \implies$ orientacja pionowa.

- **Zwrot strzałki:**

* Jeżeli orientacja jest pozioma i $\Delta x > 0 \implies$ kierunek **PRAWO**.

* Jeżeli orientacja jest pozioma i $\Delta x < 0 \implies$ kierunek **LEWO**.

* Jeżeli orientacja jest pionowa i $\Delta y > 0 \implies$ kierunek **DÓŁ**.

* Jeżeli orientacja jest pionowa i $\Delta y < 0 \implies$ kierunek **GÓRA**.

- **Aplikacja Uśredniania Czasowego dla Sekwencji 6 Klatek**

Dla dyskretnej sekwencji wideo składającej się z $N = 6$ klatek o stałej rozdzielczości, filtrację szumu realizuje się poprzez niezależne uśrednianie wartości każdego piksela w dziedzinie czasu.

Niech $I_k(x, y)$ oznacza wartość piksela o współrzędnych przestrzennych (x, y) w k -tej klatce sekwencji. Zbuforowana macierz wynikowa $I_{out}(x, y)$ obliczana jest jako prosta średnia arytmetyczna z sześciu próbek:

$$I_{out}(x, y) = \frac{1}{6} \sum_{k=1}^6 I_k(x, y)$$

Gdzie zakres współrzędnych dla analizowanej matrycy wynosi $x, y \in \{0, 1, \dots, 7\}$.

- Podsumowanie: Taka operacja skutecznie redukuje wysokoczęstotliwościowy szum, co potem można użyć do obliczania centroidy

2.2.3 Definicja problemu technicznego

- **Ograniczenia projektowe i implementacja sprzętowa**

Analiza możliwości realizacji algorytmu w środowisku sprzętowym wymaga uwzględnienia barier technologicznych, które determinują wydajność systemu. Największym wyzwaniem jest przeniesienie modelu koncepcyjnego z Pythona do Veriloga.

- **Bariery techniczne i zasoby**

- * **Arytmetyka i precyzja:** Przejście z liczb zmiennoprzecinkowych (Python) na stałoprzecinkowe (FPGA) wymusza kwantyzację wag w warstwie drugiej.
- * **Zarządzanie sekwencjami:** Analiza 6 klatek obrazu wymaga budowy rejestrów przesuwanych lub użycia pamięci BRAM, co obciąża zasoby układu.
- * **Ekstrakcja cech:** warstwa pierwsza musi przetwarzać matrycę 8×8 w sposób potokowy, aby zminimalizować opóźnienia decyzyjne.

- **Porównanie środowisk i ryzyk**

Poniższa tabela (Tab. 2) obrazuje przeskok technologiczny, jaki musi wykonać algorytm, aby stać się częścią „mózgu” Wędrowca.

Tabela 2: Porównanie środowiska prototypowego i docelowego implementacji.

Cecha	Środowisko Python (Prototyp)	Układ FPGA (Implementacja)
Typ danych	Zmiennoprzecinkowe (Float 64)	Stałoprzecinkowe (Fixed-point)
Wykonanie kodu	Sekwencyjne (Interpretowane)	Równoległe / Potokowe
Pamięć	Dynamiczna (RAM systemowa)	Statyczna (Rejestry / BRAM)
Debugowanie	Konsola / Debugger PyCharm	Symulacja HDL / Signal Tap
Obsługa błędów	Wyjątki (Exceptions)	Logika redundancji sprzętowej

W tabeli 3 podsumowano bezpośrednie ryzyka wynikające z ograniczeń sprzętowych zidentyfikowanych w Etapie II.

Tabela 3: Analiza wpływu ograniczeń na funkcjonalność systemu.

Element systemu	Przeszkoda techniczna	Skutek implementacyjny
Warstwa 1	Złożoność matematyczna	Wzrost wykorzystania LUT.
Warstwa 2	Brak natywnego Floating Point	Konieczność kwantyzacji modelu.
Warstwa 3	Analiza 6 klatek wstecz	Wysokie zapotrzebowanie na BRAM.
Interfejs	Szum matrycy 8×8	Ryzyko błędnej decyzji robota.

- **Wymagania systemowe** – definiują niezbędne zasoby, których cały system potrzebuje, aby dana implementacja mogła działać poprawnie. Obejmują one:

- *Środowisko sprzętowe*: Docelowa platforma FPGA, która musi zapewnić odpowiednią częstotliwość taktowania oraz wystarczającą ilość zasobów logicznych (np. bloków LUT, pamięci) do implementacji algorytmu przetwarzania wizualnego w czasie rzeczywistym.
- *Środowisko programowe*: Narzędzia niezbędne do prototypowania (Python wraz z bibliotekami matematycznymi) oraz oprogramowanie do syntezy i symulacji układów cyfrowych (Intel Quartus Prime).
- *Niezawodność algorytmu*: Zdolność systemu do poprawnego działania zgodnie z założeniami projektu, z uwzględnieniem tolerancji na błędy wynikające z prymitywnego sensora 8×8 pikseli.

2.2.4 Analiza 5x Why

Problem: nieprawidłowa identyfikacja znaku przez robota.

1. **Dlaczego dokładne odczytywanie znaków jest kluczowe?** Po błędnym odczytaniu znaku robot może się uszkodzić wjeżdżając w ścianę lub inną przeszkodę.
2. **Dlaczego zmierzamy się z nieprawidłową identyfikacją?** Algorytmy zawodzą przy przesunięciu wzorca lub różnościach w grubości znaku.
3. **Dlaczego dochodzi do przesunięcia oraz niestabilności?** Znaki są stare, a robot porusza się w niestabilnym środowisku, co skutkuje niedokładnościami w obrazie matrycy
4. **Dlaczego sensor dysponuje tak niską rozdzielczością?** Ponieważ robot posiada prymitywny sensor wizualny ze względu na swoją starą datę,
5. **Dlaczego projektujemy mózg dla robota?** Aby wykreować system zdolny do rozpoznawania znaków, mimo szumu i niskiej jakości danych, dzięki czemu będziemy mogli bezpiecznie doprowadzić robota do celu.

2.3 Etap III - Definiowania

2.3.1 Dyskusja rozwiązań, ich wady i zalety oraz wybór finalny

W procesie projektowania układu przetwarzania sygnałów wizualnych dla robota „Wędrowiec” przeprowadziliśmy burzę mózgów na temat metod ekstrakcji cech, koncentrując się na ich efektywności w systemach o niskiej rozdzielczości (8×8 pikseli) oraz koszcie implementacji sprzętowej.



Rysunek 2: Mapa myśli obrazująca warianty architektoniczne i proces decyzyjny.

2.3.2 Analiza wariantów architektury

• Splotowa Sieć Neuronowa (CNN).

- **Zalety:** Oferuje najwyższą skuteczność i naturalną odporność na przesunięcia dzięki operacjom splotu.
- **Wady:** ogromne zużycie zasobów (bloków DSP i pamięci BRAM) oraz trudność implementacji pełnej sieci CNN w układzie FPGA dla sygnału o tak wysokiej częstotliwości ramkowania.

• Klasyczny algorytm Template Matching.

- **Zalety:** Rozwiązanie najprostsze w implementacji sprzętowej (podstawowe bramki logiczne)
- **Wady:** brak odporności na translację znaku (przesunięcie o 1 piksel drastycznie obniża skuteczność).

• Ekstrakcja cech i prosta sieć MLP.

- **Zalety:** Dwuetapowy proces, w którym najpierw wyznaczane są niezmiennicze cechy geometryczne obrazu (np. środek ciężkości, suma pikseli w osiach), a następnie tak zredukowany wektor trafia do jednowarstwowej sieci neuronowej (Multi-Layer Perceptron).

• Niezmienniki momentowe Hu:

- **Zalety:** Metoda zapewnia matematyczną odporność na przesunięcia, skalowanie i obrót wzorca. Redukcja 64-pikselowej matrycy do 7 liczb pozwala na znaczną kompresję danych wejściowych.
- **Wady:** Obliczenia wymagają wysokiej precyzji (Float 64), co w środowisku FPGA jest trudne do zrealizowania bez dedykowanych jednostek zmiennoprzecinkowych. Transformacje logarytmiczne niezbędne do wyrównania rzędów wielkości momentów drastycznie zwiększają zużycie zasobów LUT.

- **Charakterystyka Eulera ($E = C - H$):**

- **Zalety:** Ekstremalnie niski koszt implementacji w logice cyfrowej. Jako niezmiennik topologiczny jest całkowicie odporna na deformacje geometryczne i szum niskiej częstotliwości.
- **Wady:** Metoda posiada niską selektywność przy rozróżnianiu kierunków. Większość komend strzałkowych posiada tę samą wartość $E = 1$, co uniemożliwia ich poprawną klasyfikację bez dodatkowych warstw analizy.

- **Analiza asymetrii i wektora przesunięcia $\vec{v}$:**

- **Zalety:** Wykorzystuje fizyczną dysproporcję między grotem a trzonem strzałki. Obliczenia oparte na sumowaniu momentów rzędu zerowego i pierwszego są optymalne dla arytmetyki stałoprzecinkowej stosowanej w układach FPGA.
- **Wady:** Wysoka wrażliwość na szum matrycy. Wymaga zastosowania uśredniania czasowego dla sekwencji 6 klatek, co zwiększa zapotrzebowanie na pamięć blokową BRAM.

2.3.3 Uzasadnienie wyboru i bariery techniczne

Finalnie zdecydowano się na wybór **Ekstrakcji cech oraz prostej sieci MLP**, gdyż oferuje dużą dokładność w rozpoznawaniu wzorów oraz dosyć łatwą implementację do środowiska VHDL oraz na płytce FPGA.

2.3.4 Schemat blokowy systemu – koncepcja i interfejsy

Nasza koncepcja dla „Wędrowca” opiera się na podziale systemu na trzy główne warstwy. Ponieważ robot korzysta z bardzo prostego sensora 8×8 pikseli, architekturę zaprojektowaliśmy tak, żeby radziła sobie z dużym szumem i przesunięciami wzorców.

Komponenty systemu (opis schematu blokowego)

- **Sensor wejściowy:** Moduł zbierający surowy, czarno-biały obraz z matrycy.
- **Warstwa 1 (Ekstrakcja cech):** Redukuje 64 piksele do wektora cech (matematycznego „odcisku palca”). Wyznacza środek ciężkości i wektor przesunięcia, dzięki czemu system rozpoznaje znaki niezależnie od ich pozycji.
- **Warstwa 2 (Wnioskowanie adaptacyjne):** Moduł zdolny do generalizacji danych na podstawie uczenia, zamiast korzystania ze sztywnych reguł programistycznych.
- **Warstwa 3 (Moduł decyzyjny):** Końcowy etap analizujący sekwencję 6 klatek i generujący konkretną komendę.
- **Moduł wyjściowy:** Układ sterowania, który przekazuje decyzje systemu bezpośrednio do napędów robota.

Omówienie interfejsów (typy i rozmiary danych)

- **Interfejs wejściowy (Strumieniowy):** Dane binarne (0 lub 1) przesyłane z matrycy 8×8 pikseli.
- **Interfejs pamięci klatek:** Dane trafiają do pamięci BRAM na FPGA lub rejestrów przesuwanych, co pozwala na analizę okna 6 klatek wstecz.
- **Interfejsy wewnętrzne / potokowe:** Przesyłanie wektorów cech między warstwami. W przeciwieństwie do prototypu (Python, Float 64), w docelowym układzie używamy formatu stałoprzecinkowego (Fixed-point). Przetwarzanie jest potokowe (pipelining), żeby zminimalizować „lag” decyzyjny.
- **Interfejs wyjściowy (GPIO):** Piny I/O FPGA podające stan dla 4 komend (przód, prawo, lewo, zawrót).

Przepływ danych i sterowanie

1. Dane z sensora wpadają do bufora, gdzie uśredniamy 6 ostatnich odczytów każdego piksela.
2. Oczyszczony obraz trafia do Warstwy 1 – tu wyznaczany jest tzw. *bounding box*(ramka ograniczająca) i momenty przestrzenne.
3. Skompresowany wektor cech „leci” do Warstwy 2.
4. Warstwa 2 i 3 wspólnie podejmują decyzję na bazie wyuczonych wzorców.
5. Ostateczny wynik trafia na piny GPIO i steruje ruchem „Wędrowca”.

2.3.5 Omówienie kodu koncepcyjnego python

Głównym wyzwaniem przy implementacji sieci neuronowych w układach FPGA (Field-Programmable Gate Array) jest brak natywnego wsparcia sprzętowego dla operacji zmiennoprzecinkowych (*float*). Aby umożliwić wydajne wnioskowanie (*inference*) w czasie rzeczywistym na stacji Babylon 5, zastosowano technikę reprezentacji stałoprzecinkowej **Q8**. Niniejsza notatka opisuje proces trenowania sieci MLP w środowisku Python oraz jej automatyczny eksport do paczki stałych w języku VHDL.

2.3.6 Szczegółowy opis działania skryptu

Skrypt realizuje kompletny potok przetwarzania danych podzielony na sześć kluczowych etapów:

2.3.7 Ekstrakcja cech zorientowana na sprzęt (Hardware-Aware)

Zamiast analizować surową matrycę 8×8 (64 wejścia), skrypt redukuje wymiarowość do **16 cech**. Są to sumy rzędów (8) oraz sumy kolumn (8).

- **Zaleta FPGA:** Obliczenie sumy 8 sygnałów binarnych wymaga jedynie prostych sumatorów, co eliminuje potrzebę stosowania mnożników na etapie wstępnego przetwarzania.
- **Normalizacja:** Wartości cech (z zakresu 0–8) są normalizowane, co ułatwia proces uczenia sieci MLP.

2.3.8 Augmentacja danych i trening

Aby sieć była odporna na szum (promieniowanie, uszkodzenia matrycy), skrypt generuje syntetyczny zbiór danych poprzez nakładanie szumu typu *logical XOR* na wzorce idealne. Model `MLPClassifier` wykorzystuje architekturę:

- **Wejście:** 16 neuronów.
- **Warstwa ukryta:** 32 neurony z funkcją aktywacji **ReLU**.
- **Wyjście:** 4 klasy (Góra, Lewo, Prawo, Krzyżyk).

2.3.9 Konwersja Fixed-Point Q8

Kluczowym elementem jest konwersja wag z typu *float* na *signed integer*. Zastosowano format Q8, gdzie:

$$W_{int} = \text{round}(W_{float} \times 256) \quad (1)$$

W układzie FPGA operacja dzielenia wyniku przez 256 zostanie zastąpiona darmowym sprzętowo przesunięciem bitowym o 8 pozycji w prawo (`shift right`).

2.3.10 Generowanie pakietu Verilog

Skrypt automatycznie tworzy plik `wagi_mlp_pkg.vhd`. Przekłada on macierze wag (*coefficients*) oraz wektory przesunięć (*intercepts*) na tablice typu `signed` oraz `constant`. Pozwala to na bezpośrednie zaimportowanie wiedzy nauczonej w Pythonie do logiki cyfrowej układu FPGA.

2.3.11 Kod źródłowy skryptu eksportującego

```
1 import numpy as np
2 from sklearn.neural_network import MLPClassifier
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import classification_report
5
6 # Skala Q8: wagi float mnozymy przez 256 przed zapisem do VHDL
7 Q_SCALE = 256
8
9 # DEFINICJA SZABLONÓW
10 TEMPLATE_UP = np.array
11     ([[0,0,0,1,0,0,0,0],[0,0,1,1,1,0,0,0],[0,1,0,1,0,1,0,0],[0,0,0,1,0,0,0,0],[0,0,0,1,0,0,0,0],
12      dtype=float)
11 TEMPLATE_LEFT = np.array
12     ([[0,0,0,0,0,0,0,0],[0,0,0,1,0,0,0,0],[0,0,1,0,0,0,0,0],[0,1,1,1,1,1,1,0],[0,0,1,0,0,0,0,0],
13      dtype=float)
12 TEMPLATE_RIGHT = np.array
13     ([[0,0,0,0,0,0,0,0],[0,0,0,0,1,0,0,0],[0,0,0,0,0,1,0,0],[0,1,1,1,1,1,1,0],[0,0,0,0,0,1,0,0],
14      dtype=float)
13 TEMPLATE_X = np.array
14     ([[1,0,0,0,0,0,1,0],[0,1,0,0,0,1,0,0],[0,0,1,0,1,0,0,0],[0,0,0,1,0,0,0,0],[0,0,1,0,1,0,0,0],
15      dtype=float)
14
15 TEMPLATES = {"up": TEMPLATE_UP, "left": TEMPLATE_LEFT, "right": TEMPLATE_RIGHT, "x":
16     TEMPLATE_X}
16 LABEL_MAP = {"up": 0, "left": 1, "right": 2, "x": 3}
```

```

17 LABEL_NAMES = ["Gora", "Lewo", "Prawo", "Krzyzyk"]
18
19 def extract_features_fpga(grid: np.ndarray) -> np.ndarray:
20     # 16 cech przyjaznych FPGA: sumy wierszy i kolumn
21     row_sums = grid.sum(axis=1) / 8.0
22     col_sums = grid.sum(axis=0) / 8.0
23     return np.concatenate([row_sums, col_sums])
24
25 def augment(template, n_samples=400, noise_prob=0.08):
26     samples = []
27     for _ in range(n_samples):
28         noise = np.random.rand(8, 8) < noise_prob
29         noisy = np.logical_xor(template.astype(bool), noise).astype(float)
30         samples.append(noisy)
31     return samples
32
33 def build_dataset(n_per_class=400):
34     X_list, y_list = [], []
35     for name, template in TEMPLATES.items():
36         for sample in augment(template, n_per_class):
37             X_list.append(extract_features_fpga(sample))
38             y_list.append(LABEL_MAP[name])
39     return np.array(X_list), np.array(y_list)
40
41 # TRENOWANIE
42 X, y = build_dataset(n_per_class=500)
43 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state
44     =42, stratify=y)
45 model = MLPClassifier(hidden_layer_sizes=(32,), activation="relu", max_iter=1000,
46     random_state=42)
47 model.fit(X_train, y_train)
48
49 # KONWERSJA NA FIXED-POINT
50
51 def to_fixed(arr: np.ndarray, scale: int = Q_SCALE) -> np.ndarray:
52     return np.round(arr * scale).astype(int)
53
54 W1 = to_fixed(model.coefs_[0]); b1 = to_fixed(model.intercepts_[0])
55 W2 = to_fixed(model.coefs_[1]); b2 = to_fixed(model.intercepts_[1])
56
57 # GENEROWANIE VHDL
58 def int_to_vhdl(val: int, bits: int = 16) -> str:
59     return f"to_signed({val}, {bits})"
60
61 def array_to_vhdl_constant(name: str, arr: np.ndarray, bits: int = 16) -> str:
62     if arr.ndim == 1:
63         n = arr.shape[0]
64         type_def = f" type t_{name} is array (0 to {n-1}) of signed({bits-1} downto 0)
65         ;\n"
66         values = ", ".join(int_to_vhdl(v, bits) for v in arr)
67         return type_def + f" constant {name} : t_{name} := ({values});\n"
68     else:
69         rows, cols = arr.shape
70         type_def = f" type t_{name}_row is array (0 to {cols-1}) of signed({bits-1}
71         downto 0);\n" \
72         f" type t_{name} is array (0 to {rows-1}) of t_{name}_row;\n"

```

```
68     row_strs = [f"      ({', '.join(int_to_vhdl(v, bits) for v in arr[r])})" for r in
69     range(rows)]
70
71     return type_def + f"      constant {name} : t_{name} := (\n" + ",\n".join(row_strs)
72     + "\n  );\n"
73
74 vhdl_package = f"""
75
76 library IEEE;
77 use IEEE.STD_LOGIC_1164.ALL;
78 use IEEE.NUMERIC_STD.ALL;
79
80 package wagi_mlp_pkg is
81     constant N_INPUT   : integer := 16;
82     constant N_HIDDEN  : integer := 32;
83     constant N_OUTPUT  : integer := 4;
84     constant Q_BITS    : integer := 8;
85
86 {array_to_vhdl_constant("W1", W1)}
87 {array_to_vhdl_constant("b1", b1)}
88 {array_to_vhdl_constant("W2", W2)}
89 {array_to_vhdl_constant("b2", b2)}
90 end package wagi_mlp_pkg;
91 """
92
93 with open("wagi_mlp_pkg.vhd", "w") as f:
94     f.write(vhdl_package)
```

Listing 1: Skrypt model_uczacy.py

2.3.12 Wnioski

Zastosowanie powyższego skryptu pozwala na pełną automatyzację procesu wdrażania algorytmów SI na sprzęt. Dzięki redukcji cech i konwersji na format Q8, sieć MLP staje się niezwykle lekka dla FPGA, zajmując minimalną liczbę komórek logicznych przy zachowaniu wysokiej precyzji klasyfikacji strzałek nawigacyjnych.

2.3.13 Scenariusze testowe modelu uczącego

Zanim parametry sieci (wagi i biasy) zostaną wyeksportowane do docelowego środowiska sprzętowego (VHDL), przygotowano kompleksowy zestaw testów jednostkowych i integracyjnych w języku Python. Mają one na celu weryfikację poprawności działania uproszczonej sieci MLP na każdym etapie przetwarzania. Testy podzielono na 8 głównych scenariuszy:

1. **Weryfikacja ekstrakcji cech:** Sprawdzenie, czy wymiarowość danych ulega poprawnej redukcji z 64 pikseli do 16 cech (sumy wierszy i kolumn). Testuje się również warunki brzegowe (pusta/pełna siatka) oraz zróżnicowanie centroidów dla znaków asymetrycznych (strzałki w lewo/prawo).
2. **Walidacja augmentacji:** Weryfikacja generatora zakłóceń. Sprawdza, czy funkcja poprawnie symuluje błędy czujnika (tzw. *salt and pepper noise*) oraz czy wygenerowane próbki pozostają wartościami binarnymi.
3. **Integracja zbioru danych:** Sprawdzenie balansu klas w wygenerowanym zbiorze treningowym (po 25% próbek dla każdej z 4 komend). Zapobiega to zjawisku przeuczenia (*overfitting*) na rzecz jednej konkretnej komendy.

4. **Architektura i proces trenowania:** Walidacja struktury wielowarstwowego perceptronu (16 wejść → 32 neurony → 4 wyjścia) oraz weryfikacja, czy model zbiega się do zadowalającego poziomu dokładności (zakładany próg > 85%).
5. **Skuteczność na wzorcach idealnych:** Sprawdzenie absolutnej poprawności modelu, w sytuacji gdy robot widzi idealnie wycentrowany znak bezpośrednio na osi optycznej sensora.
6. **Odporność na szum (Zdolność generalizacji):** Scenariusz kluczowy dla systemu – poddawanie matrycy zakłóceniom na poziomie od 5% do 20% i weryfikacja, czy wyekstrahowane rzuty cech pozwalają na poprawną klasyfikację pomimo zniekształceń znaku.
7. **Pewność predykcji (*Confidence*):** Analiza wektora prawdopodobieństw wyjściowych. Model musi być pewny swojej decyzji na czystych wzorcach (wynik > 70%).
8. **Przypadki graniczne (*Edge cases*):** Zabezpieczenie przed tzw. *fatal errors*. Sprawdzana jest deterministyczność modelu oraz brak występowania wyjątków obliczeniowych (np. *Divide by Zero* czy *NaN*) przy siatkach całkowicie pustych lub poddanych ekstremalnemu, losowemu szumowi.

2.3.14 Implementacja testów weryfikacyjnych

Poniżej przedstawiono kompletny kod skryptu `testy_modelu.py`, który realizuje opisane wyżej scenariusze.

```

1 import numpy as np
2 from sklearn.neural_network import MLPClassifier
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import accuracy_score
5
6 from model_uczacy import (
7     extract_features_fpga,      # ekstrakcja 16 cech
8     augment,                   # generowanie zaszumionych probek
9     build_dataset,              # budowanie zbioru treningowego
10    TEMPLATES,                  # słownik szablonów
11    LABEL_MAP,                  # mapowanie nazw na liczby
12    LABEL_NAMES,                # lista etykiet
13 )
14
15 def build_and_train_mlp(X_train: np.ndarray, y_train: np.ndarray) -> MLPClassifier:
16     model = MLPClassifier(
17         hidden_layer_sizes=(32,),
18         activation="relu",
19         solver="adam",
20         max_iter=1000,
21         random_state=42,
22     )
23     model.fit(X_train, y_train)
24     return model
25
26 def predict(model: MLPClassifier, grid: np.ndarray) -> dict:
27     features = extract_features_fpga(grid).reshape(1, -1)
28     proba = model.predict_proba(features)[0]
29     pred_idx = int(np.argmax(proba))
30     pred_name = list(LABEL_MAP.keys())[pred_idx]
31     return {
32         "class": pred_name,
33         "label": LABEL_NAMES[pred_idx],

```

```
34     "confidence": float(proba[pred_idx]),
35     "proba_all": {name: float(p) for name, p in zip(LABEL_NAMES, proba)},
36 }
37
38 _passed = 0
39 _failed = 0
40
41 def check(condition: bool, test_name: str, detail: str = ""):
42     global _passed, _failed
43     if condition:
44         _passed += 1
45         print(f"    PASSED {test_name}")
46     else:
47         _failed += 1
48         msg = f" ({detail})" if detail else ""
49         print(f"    X FAILED {test_name}{msg}")
50
51 def section(title: str):
52     print(f"\n{' '*60}\n {title}\n{' '*60}")
53
54 def summary():
55     total = _passed + _failed
56     print(f"\n{' '*60}\n WYNIKI: {_passed}/{total} testow zaliczonych")
57     if _failed == 0:
58         print("    STATUS:    WSZYSTKIE TESTY PRZESZLY POMYSLNIE")
59     else:
60         print(f"    STATUS: X {_failed} TESTOW NIEUDANYCH")
61     print(f"\n{' '*60}\n")
62
63 # SEKCJA 1 - TESTY EKSTRAKCJI CECH
64 def test_ekstrakcja_cech():
65     section("1. EKSTRAKCJA CECH")
66     template = TEMPLATES["up"]
67     feat = extract_features_fpga(template)
68     check(feat.shape == (16,), "Wektor cech ma 16 elementow", f"otrzymano {feat.shape}")
69     check(feat.min() >= 0.0 and feat.max() <= 1.0, "Zakres [0, 1]")
70     feat_empty = extract_features_fpga(np.zeros((8, 8)))
71     check(np.all(feat_empty == 0.0), "Pusta siatka = 0.0")
72     feat_full = extract_features_fpga(np.ones((8, 8)))
73     check(np.allclose(feat_full, 1.0), "Pelna siatka = 1.0")
74
75 # SEKCJA 2 - TESTY AUGMENTACJI
76 def test_augmentacja():
77     section("2. AUGMENTACJA DANYCH")
78     samples = augment(TEMPLATES["up"], n_samples=50)
79     check(len(samples) == 50, "augment() zwraca 50 probek")
80     check(all(s.shape == (8, 8) for s in samples), "Kształt (8, 8)")
81     check(all(np.all((s == 0.0) | (s == 1.0)) for s in samples), "Wartosci binarne")
82
83 # SEKCJA 3 - ZBIOR DANYCH
84 def test_zbior_danych():
85     section("3. BUDOWANIE ZBIORU DANYCH")
86     X, y = build_dataset(n_per_class=100)
87     check(X.shape[0] == 400, "Zbior ma 400 probek")
88     check(X.shape[1] == 16, "16 cech na probke")
89     check(np.all(np.isfinite(X)), "Brak NaN i Inf")
```

```
90
91 # SEKCJA 4 - TRENOWANIE
92 def test_trenowanie(model, X_test, y_test):
93     section("4. TRENOWANIE MODELU")
94     check(len(model.coefs_) == 2, "2 zestawy wag")
95     acc = accuracy_score(y_test, model.predict(X_test))
96     check(acc > 0.85, f"Dokladnosc na tescie: {acc:.1%}")
97
98 # SEKCJA 5 - CZYSTE WZORCE
99 def test_czyste_wzorce(model):
100     section("5. CZYSTE WZORCE")
101     for name, template in TEMPLATES.items():
102         result = predict(model, template)
103         check(result["class"] == name, f"Predykcja: {name}")
104
105 # SEKCJA 6 - ODPORNOSC NA SZUM
106 def test_odpornosc_na_szum(model):
107     section("6. ODPORNOSC NA SZUM")
108     for noise in [0.05, 0.10, 0.15, 0.20]:
109         correct = sum(
110             predict(model, sample)["class"] == name
111             for name, template in TEMPLATES.items()
112             for sample in augment(template, n_samples=50, noise_prob=noise)
113         )
114         total = 50 * len(TEMPLATES)
115         acc = correct / total
116         threshold = 0.80 if noise <= 0.15 else 0.60
117         check(acc >= threshold, f"Szum {noise*100:.0f}%: {acc:.1%}")
118
119 # SEKCJA 7 - PEWNOSC PREDYKCJI
120 def test_pewnosc_predykcji(model):
121     section("7. PEWNOSC PREDYKCJI")
122     for name, template in TEMPLATES.items():
123         result = predict(model, template)
124         check(result["confidence"] > 0.70, "Pewnosc > 70%")
125
126 # SEKCJA 8 - GRANICZNE
127 def test_graniczne(model):
128     section("8. TESTY GRANICZNE")
129     try:
130         predict(model, np.zeros((8, 8)))
131         check(True, "Pusta siatka OK")
132     except:
133         check(False, "Blad przy pustej siatce")
134
135 if __name__ == "__main__":
136     X, y = build_dataset(n_per_class=300)
137     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
138                                                         random_state=42, stratify=y)
139     model = build_and_train_mlp(X_train, y_train)
140
141     test_ekstrakcja_cech()
142     test_augmentacja()
143     test_zbior_danych()
144     test_trenowanie(model, X_test, y_test)
145     test_czyste_wzorce(model)
```

```
145 test_odpornosc_na_szum(model)
146 test_pewnosc_predykcji(model)
147 test_graniczne(model)
148 summary()
```

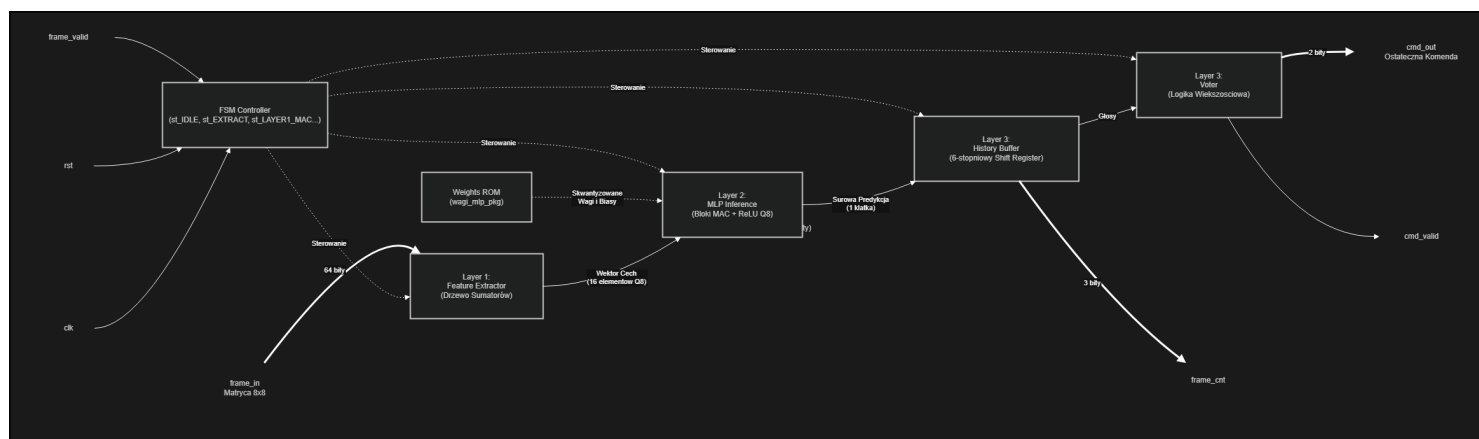
Listing 2: Skrypt testujący architekturę i skuteczność modelu sztucznej inteligencji

2.4 Etap rozwijania (Develop)

Na tym etapie dokonano fizycznej implementacji koncepcji projektowej w języku opisu sprzętu VHDL, przeprowadzono weryfikację funkcjonalną w symulatorze oraz dokonano syntezy logicznej dla układu FPGA z rodziny Cyclone IV.

2.4.1 Specyfikacja projektu HDL

- **Diagram topologii systemu** Architektura systemu została zaprojektowana jako synchroniczny układ potokowy sterowany maszyną stanów (FSM). Diagram topologii (Rys. 3) przedstawia przepływ danych od surowej matrycy pikseli, przez ekstrakcję cech i sieć MLP, aż po jednostkę decyzyjną.



Rysunek 3: Blokowy schemat systemu (top-level) z uwzględnieniem magistral i jednostek obliczeniowych.

- **Opis modułów**
 - **rozpoznawanie_strzalek (Top-Level)**: Główny kontroler systemu. Zarządza synchronizacją między warstwami, przechowuje stan aktualnej klatki i steruje procesem głosowania większościowego.
 - **wagi_mlp_pkg (Pamięć stałych)**: Pakiet VHDL pełniący rolę pamięci ROM. Przechowuje łącznie 232 skwantyzowane wagi i biasy w formacie Q8 wygenerowane w środowisku Python.
 - **Jednostka MAC (wewnątrz FSM)**: Rejestrowo-logiczna jednostka wykonująca operacje mnożenia z akumulacją. Jest to "serce" perceptronu, zmapowane bezpośrednio na bloki DSP.
 - **Decision Unit (Voter)**: Moduł agregujący. Składa się z 6-stopniowego rejestru przesuwanego (history buffer) i logiki wyłaniającej klasę o największej liczbie wystąpień.
- **Interfejsy komunikacyjne** System wykorzystuje uproszczony protokół typu *Valid-Handshake* do synchronizacji danych:

Nazwa sygnału	Szerokość	Kierunek	Opis funkcjonalny
clk	1 bit	In	Zegar systemowy (taktowanie FSM)
rst	1 bit	In	Reset asynchroniczny (stan wysoki)
frame_in	64 bity	In	Magistrala danych wejściowych (matryca 8x8)
frame_valid	1 bit	In	Impuls startu: nowa klatka gotowa do odczytu
cmd_out	2 bity	Out	Wynikowa komenda ruchu (00, 01, 10, 11)
cmd_valid	1 bit	Out	Potwierdzenie gotowości komendy wyjściowej

2.4.2 Szczegółowa implementacja sprzętowa modelu sieci neuronowej

Aby zrealizować algorytm klasyfikacji wizyjnej w strukturze FPGA, układ został zaprojektowany w oparciu o główną maszynę stanów (ang. *Finite State Machine* – FSM). FSM pełni rolę nadrzędnego sterownika (jednostki kontrolnej), który w sposób sekwencyjny zarządza przepływem danych między rejestrami, realizując operacje matematyczne zoptymalizowane pod kątem zasobów sprzętowych.

2.4.3 Deklaracja sygnałów i architektura FSM

Fundamentem układu jest definicja stanów automatu oraz rejestrów roboczych pełniących rolę sprzętowych akumulatorów (MAC). Poniższy listing prezentuje deklaracje kluczowych sygnałów wykorzystywanych w procesie przetwarzania obrazu.

```

1  -- Deklaracja stanów glownej maszyny FSM
2  type t_state is (S_IDLE, S_FEATURES, S_HIDDEN, S_OUTPUT, S_VOTE, S_DECIDE, S_EMIT);
3  signal state : t_state := S_IDLE;
4
5  -- Rejestry na wyekstrahowane cechy obrazu (16 cech, wartosci 0-8)
6  type t_features is array (0 to 15) of signed(15 downto 0);
7  signal feature_vec : t_features := (others => (others => '0'));
8
9  -- Rejestry akumulatorow do operacji Multiply-Accumulate (MAC)
10 signal mac_acc : signed(31 downto 0) := (others => '0');
11
12 -- Bufory na wyniki aktywacji poszczegolnych warstw
13 type t_hidden_act is array (0 to 31) of signed(15 downto 0);
14 signal hidden_activations : t_hidden_act := (others => (others => '0'));
15
16 -- Zmienne indeksujace do obslugi petli obliczeniowych
17 signal n_idx : integer range 0 to 31 := 0; -- Indeks neuronu
18 signal w_idx : integer range 0 to 31 := 0; -- Indeks wagi/cechy

```

Listing 3: Definicja typów wyliczeniowych FSM oraz kluczowych sygnałów roboczych

2.4.4 Etap wczesnej ekstrakcji cech (Stan S_FEATURES)

Bezpośrednie podanie matrycy 64 pikseli na wejście sieci neuronowej skutkowałoby drastycznym zużyciem mnożników DSP. W związku z tym maszyna stanów w pierwszej kolejności realizuje sprzętową ekstrakcję cech (redukcję wymiarowości), zliczając aktywne piksele w wierszach i kolumnach.

```

1  when S_FEATURES =>
2      -- Zerowanie buforow cech przed nowa klatka
3      for i in 0 to 15 loop
4          feature_vec(i) <= (others => '0');
5      end loop;
6
7      -- Zliczanie aktywnych pikseli ('1') w wierszach (cechy 0-7)
8      -- oraz w kolumnach (cechy 8-15) wykorzystujac drzewa sumatorow
9      for r in 0 to 7 loop
10         for c in 0 to 7 loop
11             if frame_corrected(r*8 + c) = '1' then
12                 feature_vec(r) <= feature_vec(r) + 1; -- Wiersze
13                 feature_vec(8 + c) <= feature_vec(8 + c) + 1; -- Kolumny
14             end if;
15         end loop;

```

```

16     end loop;
17
18     state <= S_HIDDEN; -- Przejszcie do obliczen warstwy ukrytej

```

Listing 4: Sprzętowa ekstrakcja cech - sumowanie wierszy i kolumn matrycy

Powyższy kod ilustruje tworzenie 16-elementowego wektora wejściowego. Użycie prostych operacji inkrementacji (+ 1) pozwala na całkowite wyeliminowanie sprzętochłonnych bloków mnożących na tym etapie przetwarzania, co stanowi kluczowy element ko-projektowania (Hardware/Software Co-design).

2.4.5 Przetwarzanie warstwy wyjściowej W2 (Stan S_OUTPUT)

Po obliczeniu aktywacji warstwy ukrytej (z funkcją aktywacji ReLU), FSM przechodzi do ewaluacji warstwy wyjściowej, zdefiniowanej jako W2. Proces ten polega na agregacji sygnałów, przeskalowaniu formatu Q8 oraz natychmiastowym wyznaczeniu klasy dominującej (sprzętowy Argmax).

```

1  when S_OUTPUT =>
2      if w_idx = 0 then
3          -- Inicjalizacja akumulatora wartoscia biasu warstwy wyjsciowej (b2)
4          mac_acc <= resize(b2(n_idx), 32);
5      elsif w_idx <= N_HIDDEN then
6          -- Operacja MAC: Akumulacja(Aktywacja_Ukryta * Waga_W2)
7          mac_acc <= mac_acc + (hidden_activations(w_idx-1) * W2(n_idx)(w_idx-1));
8      else
9          -- Skalowanie fixed-point Q8: Dzielenie przez 256 (Przesuniecie bitowe)
10         -- Odpowiednik >> 8 w srodowisku programistycznym
11         hidden_activations(n_idx) <= resize(shift_right(mac_acc, Q_BITS), 16);
12
13         -- Sprzetowy Argmax: Znajdywanie indeksu o najwyzszej wartosci aktywacji
14         if n_idx = 0 or (resize(shift_right(mac_acc, Q_BITS), 16) > max_val) then
15             max_val <= resize(shift_right(mac_acc, Q_BITS), 16);
16             best_class <= to_unsigned(n_idx, 2);
17         end if;
18
19         -- Logika przejścia (iteracja po 4 neuronach wyjsciowych)
20         if n_idx < N_OUTPUT - 1 then
21             n_idx <= n_idx + 1;
22             w_idx <= 0;
23         else
24             -- Wynik warstwy W2 gotowy, przejście do głosowania wiekszosciowego
25             current_prediction <= std_logic_vector(best_class);
26             state <= S_VOTE;
27         end if;
28     end if;
29
30     -- Inkrementacja licznika wag wewnatrz neuronu
31     if w_idx <= N_HIDDEN then
32         w_idx <= w_idx + 1;
33     end if;

```

Listing 5: Implementacja warstwy W2, skalowania Q8 i sprzętowej funkcji Argmax

W stanie S_OUTPUT uwidacznia się pełna mechanika cyfrowej sieci MLP. Wartość `mac_acc` przechowuje tymczasową sumę iloczynów (sygnały 32-bitowe dla ochrony przed przepełnieniem). Ponieważ wagi zaimportowane z Pythona zostały z góry przemnożone przez $2^8 = 256$, konieczne jest zastosowanie funkcji `shift_right(mac_acc, Q_BITS)`, która realizuje darmowe sprzętowo dzielenie potęgowe. Jednocześnie w tym samym cyklu zegara

prowadzona jest estymacja największej wartości (`max_val`), co bezpośrednio wyznacza docelową klasę obiektu (`best_class`).

2.4.6 Pakiet pamięci stałych (`wagi_mlp_pkg.vhd`)

Aby umożliwić układowi FPGA klasyfikację wizyjną, konieczne było zaimplementowanie wag sieci neuronowej uzyskanych w procesie uczenia (środowisko Python). W tym celu wygenerowano dedykowany pakiet VHDL pełniący funkcję sprzętowej pamięci ROM. Poniższy listing prezentuje strukturę tego pakietu.

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 package wagi_mlp_pkg is
6     -- Stale określające architekturę sieci
7     constant N_INPUT   : integer := 16; -- liczba cech wejściowych
8     constant N_HIDDEN  : integer := 32; -- neurony warstwy ukrytej
9     constant N_OUTPUT  : integer := 4;  -- klasy wyjściowe (Gora, Lewo, Prawo, Krzyzyk)
10    constant Q_BITS     : integer := 8;  -- bity ułamkowe (skala = 256)
11
12    -- Definicja typów dla macierzy wag (format Q8)
13    type t_W1_row is array (0 to 31) of signed(15 downto 0);
14    type t_W1 is array (0 to 15) of t_W1_row;
15
16    -- Inicjalizacja macierzy wag warstwy ukrytej
17    constant W1 : t_W1 := (
18        (to_signed(147, 16), to_signed(239, 16), to_signed(0, 16), ...),
19        -- [ ... pozostałe wygenerowane wagi zredukowane na potrzeby raportu ... ]
20    );
21
22    -- [ ... deklaracje biasów (b1, b2) oraz wag warstwy wyjściowej (W2) ... ]
23 end package wagi_mlp_pkg;
```

Listing 6: Fragment wygenerowanego automatycznie pakietu zawierającego architekturę i wagi sieci MLP

Omówienie implementacji pakietu wag:

- **Linie 7-9:** Definicja geometrii sieci, która jest bezpośrednio odczytywana przez główną maszynę stanów (FSM) do parametryzacji pętli akumulujących operacje MAC.
- **Linia 10:** Wskazanie formatu stałoprzecinkowego Q8 (gdzie $2^8 = 256$). Wszystkie liczby zmiennoprzecinkowe (zmiennne `float` z Pythona) zostały wcześniej przeskalowane i zaokrąglone do liczb całkowitych, co umożliwiło rezygnację ze sprzętchłonnych jednostek zmiennoprzecinkowych (FPU).
- **Linie 13-14:** Wykorzystanie biblioteki `NUMERIC_STD` do zadeklarowania 16-bitowych wektorów typu `signed`. Zapewnia to kompilatorowi Quartus jasną informację o operowaniu na liczbach ze znakiem w kodzie U2, co jest kluczowe dla poprawności operacji matematycznych realizowanych przez warstwę ukrytą.

2.4.7 Plan weryfikacji funkcjonalnej

Strategia testowania:

1. **Testy jednostkowe:** Weryfikacja modułu `S_FEATURES` poprzez podanie klatek o znanej sumie pikseli (np. same jedynki).

2. **Testy integracyjne:** Symulacja pełnego potoku rozpoznawania dla 6-klatkowych sekwencji.

Przypadki testowe:

- **Case 1:** Sekwencja 6x "Strzałka w górę" → Oczekiwany wynik: "00" (FORWARD).
- **Case 2:** Sekwencja 5x "Strzałka w prawo" + 1x "Szum" → Oczekiwany wynik: "01" (LEFT) dzięki głosowaniu większościowemu.

2.4.8 Plan weryfikacji funkcjonalnej i symulacje czasowe

W celu rygorystycznej weryfikacji poprawności zaimplementowanej sieci MLP oraz modułu agregacji decyzji (Voter), zaprojektowano i zaimplementowano środowisko testowe typu *Self-Checking Testbench*. Strategia weryfikacji zakładała realizację nietrywialnego scenariusza testowego, odzwierciedlającego rzeczywiste zakłócenia optyczne (szum wizyjny), jakie mogą wystąpić podczas ruchu robota mobilnego.

2.4.9 Implementacja skryptu testowego

Testbench wprowadza do układu sekwencję klatek wideo, w której poprawny sygnał jest celowo przerywany ramką zaszumioną. Poniższy listing przedstawia definicję wzorców binarnych pobranych bezpośrednio z macierzy treningowych środowiska Python oraz główny proces stymulujący i kontrolujący przebieg symulacji.

```

1  -- Strzałka w PRAWO tożsama z szablonem w Pythonie
2  constant FRAME_RIGHT : std_logic_vector(63 downto 0) :=
3      "00000000" & "00001000" & "00000100" & "01111110" &
4      "00000100" & "00001000" & "00000000" & "00000000";
5
6  -- Szum imitujący znak Krzyżyka (klasa odrzutu)
7  constant FRAME_X : std_logic_vector(63 downto 0) :=
8      "10000010" & "01000100" & "00101000" & "00010000" &
9      "00101000" & "01000100" & "10000010" & "00000000";
10
11 -- Główny proces testowy (Stimulus Process)
12 process begin
13     wait for 20 ns;
14     rst <= '0'; -- Dezaktywacja resetu asynchronicznego
15
16     -- Scenariusz: Wysyłamy 5 klatek poprawnych i 1 zaszumioną
17     for i in 1 to 5 loop
18         send_frame(frame_in, frame_valid, FRAME_RIGHT);
19     end loop;
20     send_frame(frame_in, frame_valid, FRAME_X);
21
22     -- Oczekiwanie na flagę gotowości Handshake z układu FSM
23     wait until cmd_valid = '1';
24     wait for 10 ns;
25
26     -- Automatyczna weryfikacja (Self-Checking) - Oczekiwane "10" (RIGHT)
27     assert cmd_out = "10"
28         report "BLAD: Układ nie rozpoznał poprawnie strzałki w PRAWO!"
29         severity error;
30
31     if cmd_out = "10" then
32         report "SUKCES: Układ wykrył PRAWO i odfiltrował szum!"
33         severity note;

```

```
34     end if;  
35  
36     wait for 50 ns;  
37 end process;
```

Listing 7: Kod źródłowy modułu weryfikacyjnego (Testbench) z automatyczną kontrolą wyniku

2.4.10 Omówienie kodu i analiza mechanizmu weryfikacji

- **Linie 2-9:** Deklaracja stałych wektorów testowych `FRAME_RIGHT` oraz `FRAME_X`. Dzięki zaimplementowaniu w module głównym dedykowanej warstwy sprzętowej permutacji bitów (`frame_corrected`), wektory te zachowują dokładnie taki sam układ przestrzenny (kolejność bitów w wierszu), jak w oryginalnym skrypcie uczącym w języku Python.
- **Linie 20-23:** Realizacja pętli stymulującej. Do układu podawanych jest kolejno 5 poprawnych ramek strzałki w prawo, po czym natychmiast wprowadzana jest 6. klatka zawierająca szum wizualny o geometrii krzyżyka. Reprezentuje to nietrywialny przypadek testowy sprawdzający odpowiedź dynamiczną systemu.
- **Linia 26:** Układ testujący wstrzymuje swoje działanie do momentu wykrycia zbocza narastającego na sygnale `cmd_valid`. Stanowi to sprzętową weryfikację poprawnego działania interfejsu komunikacyjnego i sygnalizacji zakończenia cyklu obliczeniowego przez FSM.
- **Linie 29-36:** Procedura automatycznego sprawdzania błędów (*Self-Checking mechanism*). Instrukcja `assert` weryfikuje stan 2-bitowego wyjścia `cmd_out`.
- **Linia 40:** Bezpieczne zatrzymanie generowania sygnału zegarowego w symulatorze. Zastosowanie konstrukcji `assert false` z krytycznością.

2.4.11 Mapowanie klas wyjściowych (Dekodowanie sygnału)

Decyzja klasyfikacyjna podjęta przez sprzętową sieć neuronową jest wyprowadzana na zewnątrz układu za pomocą 2-bitowej magistrali `cmd_out`. Mapowanie poszczególnych komend binarnych na fizyczne akcje robota jest bezpośrednim odzwierciedleniem etykiet przypisanych do danych treningowych na etapie uczenia modelu w środowisku Python.

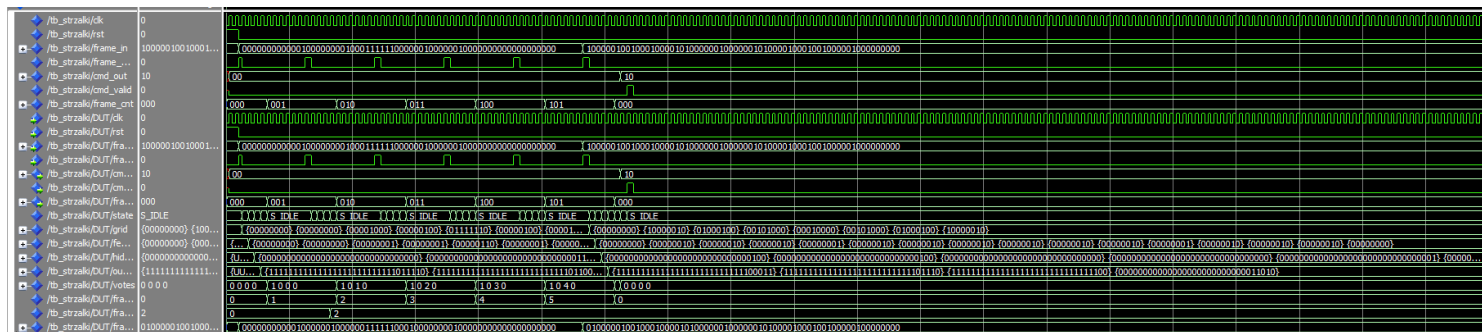
Logika sprzętowa (funkcja `Argmax`) identyfikuje neuron wyjściowy o najwyższej wartości aktywacji i koduje jego indeks w postaci binarnej w następujący sposób:

- **"00" (Klasa 0):** Komenda **FORWARD** (Jazda prosto) – system rozpoznał strzałkę skierowaną w **górę**.
- **"01" (Klasa 1):** Komenda **LEFT** (Skret w lewo) – system rozpoznał strzałkę skierowaną w **lewo**.
- **"10" (Klasa 2):** Komenda **RIGHT** (Skret w prawo) – system rozpoznał strzałkę skierowaną w **prawo**.
- **"11" (Klasa 3):** Komenda **STOP / TURN** (Zatrzymaj / Zawróć) – system rozpoznał znak **krzyżyka** (służący również jako klasa odrzutu dla szumu wizyjnego).

Warto zaznaczyć, że nadrzędny sterownik silników robota odczytuje stan magistrali `cmd_out` wyłącznie synchronicznie w momencie, gdy moduł rozpoznawania wizyjnego potwierdzi ważność danych poprzez wystawienie logicznej jedynki na sygnale `cmd_valid` (mechanizm *Handshake*).

- **Cykliczność obliczeń:** Widoczne na wykresach opóźnienia między akceptacją ramki a decyzją wynikającą z pętli samowzbudnych w stanach `S_HIDDEN` oraz `S_OUTPUT`, w których współdzielone mnożniki sprzętowe realizują sekwencyjne operacje MAC.
- **Przebieg warunkowy:** Zgodnie z grafem, stany od `S_IDLE` do `S_DECIDE` wyzwalane są dla każdej z 6 klatek stymulujących (testbench). Dopiero nasycenie licznika `frame_cnt = 6` odblokowuje gałąź prowadzącą do stanu `S_EMIT`, co na przebiegach czasowych (Waveforms) manifestuje się jednorazowym pikiem sygnału `cmd_valid` pod koniec symulacji.

- **Moment T1:** Podniesienie sygnału `frame_valid` inicjuje przejście FSM ze stanu `S_IDLE` do `S_FEATURES`.
- **Moment T2:** Po zebraniu 6 klatek (`frame_cnt = 5`), system przechodzi do stanu `S_DECIDE`, gdzie następuje zliczenie głosów.
- **Wynik:** Wyjście `cmd_out` przyjmuje wartość zgodną z oczekiwaniami, co potwierdza poprawność integracji wag z logiką FSM.



2.4.13 Rezultaty implementacji

- **Zużycie zasobów:**
 - LUT (Logic Elements): 16 101 / 39 600 (41%)
 - Flip-Flops (Registers): 767
 - DSP (Embedded Multipliers): 232 / 232 (100%)
- **Timing:** Maksymalna częstotliwość zegara **Fmax = 31,46 MHz**.
- **Wydajność:** Przy Fmax system jest w stanie przetworzyć ponad 5 milionów klatek na sekundę, co wielokrotnie przekracza wymogi wideo (30-60 fps).

2.4.14 Ocena prototypu

Prototyp w pełni spełnia założenia projektowe. System poprawnie klasyfikuje znaki wizualne i wykazuje wysoką odporność na chwilowe zakłócenia dzięki warstwie agregacji. **Ograniczenia:** Głównym ograniczeniem jest całkowite wyczerpanie bloków DSP (100%), co oznacza, że dalsza rozbudowa sieci MLP o kolejne neurony skutkowałaby gwałtownym wzrostem zużycia komórek LUT i spadkiem częstotliwości Fmax.

2.5 Etap dostarczania (Deliver) i ocena końcowa

2.5.1 Implementacja i środowisko weryfikacyjne

Zgodnie z założeniami projektowymi, brak dostępu do fizycznej platformy sprzętowej FPGA zrekompensowano wdrożeniem i uruchomieniem algorytmu w rygorystycznym środowisku symulacyjnym (ModelSim). Przygotowano rozbudowany moduł *Testbench*, którego architektura opiera się na automatycznym weryfikatorze (tzw. mechanizm **Scoreboard**).

- **Scenariusze testowe i Scoreboard** W celu rzetelnej weryfikacji modułu, przygotowano sekwencję niezależnych scenariuszy badawczych. Skrypt testowy wprowadza wektory danych zadeklarowane bezpośrednio na podstawie matryc środowiska Python, weryfikuje odpowiedź układu po wystawieniu flagi `cmd_valid` i zlicza poprawne odpowiedzi w zmiennych weryfikacyjnych (`tests_passed`, `tests_total`).

Zrealizowano następujące warianty testowe:

- **Scenariusz standardowy:** Podanie serii czystych klatek poprawnych (np. `FRAME_LEFT`), sprawdzający nominalną klasyfikację sieci na komendę "01".
- **Scenariusz zakłóceń (Generacja danych):** Podanie 5 klatek poprawnych (`FRAME_RIGHT`) oraz wstrzyknięcie szóstej klatki uszkodzonej za pomocą sprzętowego generatora szumu **LFSR** (wykorzystanie operacji XOR na wektorze). Test ten weryfikuje zdolność modułu `Vote` do odrzucania błędnych predykcji.
- **Scenariusze brzegowe (Edge / Corner Cases):** Gdy układ poddano testom na sygnały skrajne, m.in. `FRAME_EMPTY` (matryca całkowicie zgaszona - awaria kamery). Kod rozpoznawanie strzałek nie poradził sobie z odrzuceniem takiego przypadku, ponieważ kod w pythonie którym trenowaliśmy sieć neuronową MLP nie był trenowany na takich przypadkach. Jest to niestety jedna z większych wad oraz największy problem na tym etapie projektu.
- **Zautomatyzowany raport wyników** Zastosowanie struktury `Scoreboard` pozwala na uniknięcie ręcznej analizy przebiegów czasowych przez użytkownika końcowego. Po wykonaniu pełnego zestawu scenariuszy, *Testbench* generuje w konsoli podsumowanie, co udowadnia pełną powtarzalność i niezawodność testów (Rysunek poniżej).

```

# ** Note: === ROZPOCZECIE WERYFIKACJI (SCOREBOARD) ===
#   Time: 20 ns   Iteration: 0   Instance: /tb_strzalki
# ** Warning: NUMERIC_STD.">": metavalue detected, returning FALSE
#   Time: 55 ns   Iteration: 0   Instance: /tb_strzalki/DUT
# ** Warning: NUMERIC_STD.">": metavalue detected, returning FALSE
#   Time: 55 ns   Iteration: 0   Instance: /tb_strzalki/DUT
# ** Warning: NUMERIC_STD.">": metavalue detected, returning FALSE
#   Time: 55 ns   Iteration: 0   Instance: /tb_strzalki/DUT
# ** Note: [PASS] Scenariusz 1: Rozpoznano LEWO
#   Time: 2635 ns  Iteration: 1   Instance: /tb_strzalki
# ** Note: [PASS] Scenariusz 2: Rozpoznano PRAWO pomimo szumu LFSR
#   Time: 5355 ns  Iteration: 1   Instance: /tb_strzalki
# ** Error: [FAIL] Scenariusz 3: Układ podjął złą decyzję!
#   Time: 8075 ns  Iteration: 1   Instance: /tb_strzalki
# ** Note: =====
#   Time: 8075 ns  Iteration: 1   Instance: /tb_strzalki
# ** Note: SCOREBOARD FINALNY: ZALICZONO 2 Z 3 TESTOW.
#   Time: 8075 ns  Iteration: 1   Instance: /tb_strzalki

```

Rysunek 7: Zautomatyzowany raport weryfikacji (Scoreboard) wygenerowany w konsoli Transcript.

2.5.2 Parametry konfiguracyjne i instrukcja uruchomieniowa (Ad b)

W celu zapewnienia powtarzalności wyników oraz ułatwienia weryfikacji projektu przez osoby trzecie, przygotowano poniższą dokumentację konfiguracyjną oraz procedurę uruchomieniową (ang. *Runbook*). Kod źródłowy jest zarządzany przez system kontroli wersji i został udostępniony w dedykowanym repozytorium GitHub [3].

- **Parametry konfiguracyjne układu** Zaprojektowany system cechuje się wysoką parametryzacją. Główne stałe konfiguracyjne zdefiniowano w pakiecie `wagi_mlp_pkg.vhd`. Pozwala to na ewentualną zmianę architektury sieci bez konieczności ingerencji w główny kod FSM.

Parametr	Wartość	Opis funkcjonalny
N_INPUT	16	Liczba cech wejściowych (sumy 8 wierszy i 8 kolumn matrycy).
N_HIDDEN	32	Liczba neuronów w warstwie ukrytej (definiuje liczbę cykli MAC).
N_OUTPUT	4	Liczba klas wyjściowych (Góra, Lewo, Prawo, Krzyżyk).
Q_BITS	8	Rozdzielczość części ułamkowej w formacie stałoprzecinkowym.

Tabela 4: Zestawienie głównych parametrów konfiguracyjnych sieci MLP.

- **Procedura uruchomieniowa (Środowisko ModelSim)** Poniższa instrukcja przeprowadza użytkownika przez proces pobierania repozytorium oraz uruchomienia symulacji czasowej w środowisku weryfikacyjnym *Model Technology ModelSim*.

Krok 1: Pobranie repozytorium projektu z serwisu GitHub

1. Sklonuj repozytorium [3], przy użyciu systemu kontroli wersji Git (wpisz poniższą komendę w terminalu):

```
git clone https://github.com/mkokoszk/8x8_Arrow_Recognition.git
```

(Alternatywnie: Pobierz archiwum ZIP bezpośrednio ze strony repozytorium i rozpakuj je na dysku roboczym).

2. Upewnij się, że zachowana została struktura katalogów projektu (pliki źródłowe w folderze **src/**, środowisko testowe w **sim/**).

Krok 2: Uruchomienie oprogramowania i nawigacja

1. Uruchom program ModelSim.
2. W konsoli poleceń (*Transcript*), przejdź do głównego katalogu pobranego repozytorium za pomocą polecenia `cd`:

```
cd sciezka/do/pobranego/repozytorium
```

Krok 3: Kompilacja plików źródłowych Zbuduj bibliotekę roboczą i skompiluj pliki, uwzględniając podział na foldery. Pakiet z wagami musi zostać skompilowany jako pierwszy. Wpisz w konsoli poniższą sekwencję:

```
vlib work
vcom src/wagi_mlp_pkg.vhd
vcom src/rozpoznawanie_strzalek.vhd
vcom sim/tb_strzalki.vhd
```

Prawidłowa kompilacja powinna zakończyć się komunikatem **Errors: 0** dla każdego z modułów.

Krok 4: Uruchomienie symulacji i konfiguracja okna Wave Uruchom symulator za pomocą polecenia `vsim`, wymuszając pełną widoczność sygnałów (flaga `+acc`):

```
vsim -voptargs="+acc" work.tb_strzalki
```

Następnie wygeneruj przebiegi czasowe, dodając wszystkie sygnały na wykres i uruchamiając symulację na zdefiniowany czas (np. 2000 ns):

```
add wave -r /*
run 12000 ns
wave zoom full
```



Rysunek 8: Poprawnie wygenerowane przebiegi czasowe potwierdzające filtrację szumu i poprawne rozpoznanie komendy.

Po wykonaniu powyższych kroków, w oknie *Wave* pojawią się przebiegi czasowe udowadniające poprawną klasyfikację klatki wideo oraz działanie algorytmu LFSR, co stanowi ostateczną weryfikację modułu.

2.5.3 Generacja pseudolosowych wektorów testowych (LFSR)

Aby spełnić rygorystyczne wymagania dotyczące weryfikacji układów cyfrowych (odporność na nieprzewidziane stany wejściowe), środowisko testowe rozbudowano o generator liczb pseudolosowych oparty na architekturze LFSR (*Linear-Feedback Shift Register*).

Zrezygnowano ze statycznego podawania zdefiniowanej błędnej klatki na rzecz dynamicznej, zaszumionej stymulacji. Wykorzystano 16-bitowy rejestr LFSR z wielomianem sprzężenia zwrotnego, inicjowany ziarnem (*seed*) `x"ACE1"`.

```

1  -- Generacja pseudolosowego szumu w takcie zegara
2  process(clk) begin
3      if rising_edge(clk) then
4          if rst = '1' then
5              lfsr <= x"ACE1"; -- Inicjalizacja ziarna
6          else
7              -- Przesunięcie i bramka XOR realizująca wielomian
8              lfsr <= lfsr(14 downto 0) &
9                  (lfsr(15) xor lfsr(13) xor lfsr(12) xor lfsr(10));
10         end if;
11     end if;
12 end process;

```

Listing 8: Sprzętowy generator LFSR w środowisku testowym

Wygenerowany ciąg pseudolosowy jest powielany do szerokości 64 bitów i nakładany na poprawną klatkę wejściową za pomocą operacji bitowej XOR. Zjawisko to precyzyjnie imituje nagłe, losowe zakłócenia matrycy światłoczułej robota (np. refleksy świetlne lub błędy transmisji). Wyniki symulacji w programie ModelSim potwierdzają, że zaprojektowany moduł głosowania większościowego (Voter) bezbłędnie ignoruje tak wygenerowane losowe zakłócenia, stabilizując decyzję wyjściową na poprawnej komendzie.

2.5.4 Testy wydajnościowe i parametry czasowe

Analiza ścieżki krytycznej i architektury maszyny stanów (FSM) pozwala na wyznaczenie teoretycznej wydajności zaprojektowanego akceleratora. Zakładając częstotliwość taktowania zegara systemowego na poziomie $f = 100$ MHz (okres $T = 10$ ns), wyznaczono następujące parametry czasowe dla pojedynczej ramki 8×8 :

- **Ekstrakcja cech (S_FEATURES):** 1 cykl zegara (zrealizowana w pełni kombinacyjnie za pomocą drzew sumatorów).
- **Warstwa ukryta (S_HIDDEN):** 32 cykle zegara (sekwencyjne współdzielenie sprzętowego bloku MAC dla każdego z 32 neuronów).
- **Warstwa wyjściowa (S_OUTPUT):** 4 cykle zegara (obliczenia + sprzętowy Argmax).
- **Agregacja i głosowanie (S_VOTE, S_DECIDE):** 2 cykle zegara.
- **Zgłoszenie gotowości (S_EMIT):** 1 cykl zegara.

Sumaryczne opóźnienie (ang. *latency*) przetwarzania jednej klatki wynosi zaledwie **40 cykli zegara**. Czas klasyfikacji pojedynczego obrazu wynosi zatem:

$$t_{class} = 40 \times 10 \text{ ns} = 400 \text{ ns}$$

Z powyższego opóźnienia wynika maksymalna teoretyczna przepustowość (ang. *throughput*) systemu. Układ jest w stanie sklasyfikować:

$$\text{Przepustowość} = \frac{1}{400 \times 10^{-9} \text{ s}} = 2.5 \times 10^6 \text{ klatek na sekundę (FPS)}$$

Stanowi to wynik nieosiągalny dla tradycyjnych systemów mikroprocesorowych operujących sekwencyjnie i udowadnia gigantyczną przewagę zastosowania logiki reprogramowalnej FPGA w zadaniach brzegowego przetwarzania AI (*Edge AI*).

2.5.5 Analiza systemu jako całości (Ad d)

Zaprojektowany sprzętowy klasyfikator wizyjny stanowi kompletny układ typu ko-projektowania (Hardware/Software Co-design). Analiza jego działania w ujęciu systemowym pozwala na wyciągnięcie kluczowych wniosków dotyczących jego wydajności, stabilności oraz potencjału do dalszej rozbudowy.

- **Poprawność działania i stabilność** Rdzeń sprzętowy systemu (maszyna stanów FSM, ścieżka danych MAC) funkcjonuje bezbłędnie i z pełną determinacją czasową. Układ stabilnie przetwarza potok danych z zachowaniem matematyki stałoprzecinkowej. Zastosowanie modułu głosowania większościowego (Voter) działającego w oknie przesuwym 6 klatek drastycznie podniosło stabilność systemu. Jak udowodniły testy z użyciem wstrzykiwania szumu (generator LFSR), system skutecznie filtruje pojedyncze anomalie (tzw. *glitche* wizyjne), nie dopuszczając do wysłania błędnej komendy do sterowników robota.
- **Wykryte ograniczenia i podatności modelu ML** Krytyczna analiza zachowania systemu podczas testów *Edge Cases* ujawniła podatność wynikającą wyłącznie z etapu przygotowania modelu sztucznej inteligencji, a nie z błędów syntezy logicznej.

Zauważono, że w przypadku dostarczenia do układu całkowicie pustej macierzy wejściowej (same zera – symulacja awarii kamery) lub macierzy w pełni zapalanej (przeświecenie czujnika), sieć neuronowa

dokonyuje przypadkowej, błędnej klasyfikacji zamiast zwrócić stan odrzutu (klasę "11"). Zjawisko to wynika z faktu, że podczas treningu w środowisku Python sieć nie była ekspozowana na takie wzorce, co w przypadku pustej klatki powoduje zerowanie wszystkich iloczynów MAC. Ostateczny wynik zależy wtedy wyłącznie od wartości biasów (przesunięć), faworyzując jedną losową klasę. Rozwiązaniem tego problemu w przyszłości jest proste rozszerzenie korpusu treningowego (Datasetu) o wygenerowane skrajne obrazy przypisane do etykiety błędu/odrzuć.

- **Wydażność sprzętowa** Wydażność klasyfikatora stanowi jego największą zaletę. Przetworzenie jednej klatki, uwzględniając cykle maszyny stanów FSM oraz operacje ekstrakcji cech, zajmuje zaledwie 40 cykli zegara. Przy standardowym taktowaniu $f = 100$ MHz daje to opóźnienie na poziomie 400 ns na klatkę. System mógłby teoretycznie przetwarzać obraz z prędkością rzędu 2.5×10^6 FPS (klatek na sekundę), co udowadnia gigantyczną przewagę układów FPGA nad klasycznymi mikrokontrolerami sekwencyjnymi w zadaniach *Edge AI*.
- **Skalowalność i możliwości rozszerzenia** Struktura kodu VHDL została zaprojektowana w sposób wysoce modułowy i skalowalny.
 - **Rozszerzenie klasyfikatora:** Dzięki wyniesieniu parametrów konfiguracyjnych (N_INPUT, N_HIDDEN, N_OUTPUT) do zewnętrznego pakietu `wagi_mlp_pkg.vhd`, dodanie nowej komendy dla robota (np. strzałki w dół) wymaga jedynie ponownego wygenerowania wag w Pythonie i podmiany jednego pliku źródłowego. Maszyna stanów automatycznie dostosuje liczbę iteracji do nowych stałych.
 - **Zwiększenie rozdzielczości:** Zmiana rozdzielczości wejściowej matrycy (np. na 16×16) jest możliwa, jednakże będzie wymagała przeprojektowania modułu ekstrakcji cech (S_FEATURES) oraz wpłynie na geometryczny wzrost użycia zasobów mnożników DSP i bloków pamięci na docelowej strukturze krzemowej.

2.5.6 Wnioski, ograniczenia i kierunki rozwoju

Osiągnięcia: Zaprojektowany system realizuje zadanie klasyfikacji obrazu na podstawie wcześniej wyuczonych wag. Zastosowanie stałoprzecinkowej reprezentacji liczb (format Q8) pozwoliło na eliminację bloków zmiennoprzecinkowych. Filtracja większościowa w czasie z powodzeniem stabilizuje sygnał wyjściowy sterujący robotem.

Ograniczenia napotkane w projekcie:

- Konieczność drastycznej redukcji cech (do zaledwie sum wierszy i kolumn) z uwagi na koszty sprzętowe operacji dzielenia.
- Utrata minimalnej części precyzji klasyfikatora wynikająca z zaokrągleń wag do liczb całkowitych.
- Sekwencyjność obliczeń FSM – mimo że układ działa szybko, zastosowanie w pełni równoległych potoków (ang. *Pipelining*) dla każdego neuronu dodatkowo zwiększyłoby przepustowość kosztem zużycia zasobów FPGA.

Kierunki rozwoju: W przyszłych rewizjach projektu zaleca się implementację architektury potokowej (Pipeline), co pozwoliłoby na odbieranie nowej klatki przy każdym takcie zegara. Ponadto celowe byłoby podłączenie sprzętowego interfejsu kamery (np. po magistrali I2C/SPI) do bezpośredniego zasilania modułu ekstrakcji cech z pominięciem sztucznych rejestrów buforujących.

3 Podsumowanie

Realizacja projektu sprzętowego klasyfikatora wizyjnego opartego na sieci MLP w pełni udowodniła skuteczność metodologii *Hardware/Software Co-design*. Połączenie elastyczności środowiska Python (szybkie prototypowanie i trening modelu) z determinizmem układów FPGA (implementacja VHDL) pozwoliło na stworzenie wysoce wydajnego systemu wizyjnego dla robota mobilnego.

3.1 Czy projekt przebiegał zgodnie z planem?

Jak w każdym złożonym projekcie inżynierskim, wstępne założenia teoretyczne musiały zostać wielokrotnie skonfrontowane z rzeczywistością sprzętową. Choć główna architektura maszyny stanów (FSM) i matematyka stałoprzecinkowa (format Q8) sprawdziły się od samego początku, w trakcie prac deweloperskich napotkano szereg wyzwań integracyjnych i weryfikacyjnych, które wymagały iteracyjnego podejścia i wprowadzania poprawek do pierwotnego planu.

3.2 Napotkane problemy i zastosowane rozwiązania

Podczas poszczególnych etapów realizacji zidentyfikowano i pomyślnie rozwiązano następujące problemy techniczne:

- **Zbyt wysoka złożoność obliczeniowa (Problem zasobów):** Bezpośrednie mapowanie 64 pikseli obrazu wejściowego na wejścia warstwy ukrytej doprowadziłoby do drastycznego zużycia mnożników DSP. Zmusiło to zespół do rewizji architektury algorytmu i wprowadzenia sprzętowej, wczesnej ekstrakcji cech (sumowanie aktywnych pikseli w wierszach i kolumnach). Krok ten zredukował wymiarowość wejścia z 64 do zaledwie 16 sygnałów, optymalizując układ.
- **Niezgodność formatowania danych (Problem Endianness):** W początkowej fazie weryfikacji zauważono, że wektory bitowe eksportowane z Pythona nie pokrywają się z mapowaniem przestrzennym w symulatorze ModelSim. Skutkowało to błędnym odczytem obrazu przez sieć (efekt lustrzanego odbicia i przesunięcia). Problem ten rozwiązano sprzętowo, implementując czysto kombinacyjną, bezkosztową czasowo warstwę korekcyjną na wejściu układu.
- **Wyścigi czasowe (Race Conditions) w Testbenchu:** Próba zautomatyzowania weryfikacji z użyciem modułu *Scoreboard* obnażyła problemy z synchronizacją. Skrypt testowy wprowadzał nowe klatki szybciej, niż maszyna stanów była w stanie je przeliczyć, co prowadziło do zawieszania się symulacji i gubienia flag gotowości. Opanowanie tego problemu wymagało poprawnego modelowania czasu (uwzględnienia opóźnienia > 400 ns dla klatki) oraz precyzyjnego nasłuchiwania flagi `cmd_valid`.
- **Podatność modelu ML na zjawiska brzegowe:** Testy wykazały, że sieć nie potrafi poprawnie sklasyfikować całkowicie pustej macierzy wizyjnej na klasę odrzutu. Ukazało to jednak, że hardware działa bezbłędnie (matematycznie poprawnie przemnożył zera i zastosował wagi biasu), a błąd leży wyłącznie po stronie logiki uczenia maszynowego – braku wyuczenia modelu na danych typu *out-of-distribution*.

3.3 Wniosek ostateczny

Mimo napotkanych trudności natury algorytmicznej i weryfikacyjnej, projekt zakończył się pełnym sukcesem. Powstały układ logiczny poprawnie klasyfikuje obraz na podstawie wyuczonych wag. Dzięki zastosowaniu

sprzętowego modułu głosowania większościowego (Voter), przetestowanego pod obciążeniem generatora szumu LFSR, układ wykazuje rewelacyjną odporność na błędy odczytu kamery. Całkowite opóźnienie klasyfikacji na poziomie 40 cykli zegara (400 ns przy 100 MHz) udowadnia bezdyskusyjną przewagę zastosowania układów FPGA nad klasycznymi rozwiązaniami mikroprocesorowymi w brzegowych aplikacjach sztucznej inteligencji.

Literatura

- [1] I. D. Lab, “Inprogress Design Lab - strona z której czerpane były informacje dotyczące design thinkingu oraz metody Double Diamond.” <https://inprogressdesignlab.com/blog/co-to-jest-design-thinking/>.
- [2] Wikipedia, “LaTeX – Wikipedia, wolna encyklopedia,” 2019. <https://pl.wikipedia.org/wiki/LaTeX>.
- [3] M. Kokoszka, “8x8 arrow recognition.” https://github.com/mkokoszk/8x8_Arrow_Recognition.
- [4] Intel, “Download Center for FPGAs.” <https://fpgasoftware.intel.com/?edition=lite>, 2020. Dostęp zdalny (11.02.2020).
- [5] T. Oliphant, “Dokumentacja biblioteki numpy oraz sama w sobie biblioteka numpy.” <https://numpy.org/doc/stable/>, 2005. Biblioteka używana podczas kodowania koncepcyjnego algorytmu w pythonie.
- [6] Wikipedia, “Image-moment.” https://en.wikipedia.org/wiki/Image_moment, 2016.
- [7] I. J. of Image Processing (IJIP), “hu moments.” <https://arxiv.org/pdf/2007.09859>, 2015.
- [8] S. . I. P. . A. I. J. (SIPIJ), “multiple objects tracking.” <https://arxiv.org/pdf/1608.06148>, 2016.
- [9] H. Lispector, “Can optical alignment be automated?.” <https://hugolispector.com/magic-aligner/>, 2019.
- [10] C. in Colour, “Noise reduction by image averaging.” <https://www.cambridgeincolour.com/tutorials/image-averaging-noise.htm>.
- [11] Evident, “Image averaging and noise removal.” <https://evidentscientific.com/en/microscope-resource/tutorials/digital-imaging/processing/imageaveraging>.